

Day 07 – Linux File System Hierarchy & Scenario-Based Practice

Directory	Purpose	I would use this when
/	Root of the Linux filesystem; all directories start here	I need to view or navigate the entire system
/home	Stores home folders for normal users	Managing user files or SSH access
/root	Home directory of the root user	Performing admin-level tasks
/etc	System and application configuration files	Editing service or system configs
/var/log	System and application log files	Troubleshooting errors and failures
/tmp	Temporary files used by users and apps	Testing or short-term storage
/bin	Essential system command binaries	Running basic Linux commands
/usr/bin	Most user-level command binaries	Daily DevOps and user operations
/opt	Optional or third-party applications	Managing external software

```
# Find the largest log file in /var/log
du -sh /var/log/* 2>/dev/null | sort -h | tail -5

# Look at a config file in /etc
cat /etc/hostname

# Check your home directory
ls -la ~
```

- The first command finds the **largest log files** in /var/log, helping identify what is using the most disk space.
- The second command shows the system's **hostname** from a config file in /etc.
- The third command lists **all files (including hidden ones)** in your home directory, useful for checking user configs and permissions.

Scenario 2

```
ubuntu@ip-172-31-23-127:~$  
ubuntu@ip-172-31-23-127:~$ du -sh /var/log/* 2>/dev/null | sort -h | tail -5  
228K    /var/log/sysstat  
244K    /var/log/kern.log  
456K    /var/log/cloud-init.log  
892K    /var/log/syslog  
54M     /var/log/journal  
ubuntu@ip-172-31-23-127:~$ cat /etc/hostname  
ip-172-31-23-127  
ubuntu@ip-172-31-23-127:~$ ls -la ~  
total 68  
drwxr-x--- 7 ubuntu ubuntu 4096 Feb  9 17:33 .  
drwxr-xr-x 5 root    root   4096 Feb  3 09:24 ..  
-rw----- 1 ubuntu ubuntu 4410 Feb  9 16:42 .bash_history  
-rw-r--r-- 1 ubuntu ubuntu  220 Mar 31 2024 .bash_logout
```

Part 2: Scenario-Based Practice (40 minutes)

Scenario 1: Service Not Starting (myapp)

Step 1:

systemctl status myapp

Why: Checks whether the service is running, failed, or inactive and shows the immediate error reason.

Step 2:

journalctl -u myapp -n 50

Why: Reads the last 50 log entries for myapp to identify startup errors or crashes.

Step 3:

systemctl is-enabled myapp

Why: Verifies whether the service is configured to start automatically after reboot.

Step 4:

systemctl list-dependencies myapp

Why: Checks if myapp depends on other services that may have failed to start

Scenario 2: High CPU Usage

Step 1:

top

Why: Shows live CPU usage and running processes in real time to quickly spot spikes.

Step 2:

```
ps aux --sort=-%cpu | head -10
```

Why: Lists the top 10 processes sorted by CPU usage, making the biggest offender obvious.

Step 3:

```
ps -o pid,ppid,%cpu,%mem,cmd -p <PID>
```

Why: Inspects the exact process using high CPU to understand what it is and how it was started.

Step 4:

```
top -p <PID>
```

Why: Monitors only the suspected process to confirm it's consistently consuming high CPU.

Scenario 3: Finding Service Logs (docker)

Step 1:

```
systemctl status docker
```

Why: Confirms the service state and shows recent log lines plus the exact unit name.

Step 2:

```
journalctl -u docker -n 50
```

Why: Displays the last 50 log entries for the Docker service from journald.

Step 3:

```
journalctl -u docker -f
```

Why: Follows Docker logs in real time to observe live activity or errors (like tail -f).

Step 4 (optional but pro move):

```
journalctl -u docker --since "10 minutes ago"
```

Why: Filters logs by time to focus only on recent issues.

Scenario 4: File Permissions Issue (Permission denied)

Step 1: Check current permissions

Command:

```
ls -l /home/user/backup.sh
```

Why: Shows file permissions; absence of x means the script cannot be executed.

Step 2: Add execute permission

Command:

```
chmod +x /home/user/backup.sh
```

Why: Grants execute (x) permission so the script can be run.

Step 3: Verify permissions changed

Command:

```
ls -l /home/user/backup.sh
```

Why: Confirms that x permission is now present (e.g., -rwxr-xr-x).

Step 4: Run the script

Command:

```
./backup.sh
```

Why: Executes the script from the current directory to confirm the issue is resolved.